

Dokumentation “MLMC” Project

Das Projekt ist grundsätzlich in 4 Teile zu unterteilen. Alle Programme können ohne weitere Eingaben direkt gestartet werden. Einzige Ausnahme ist eine „workers“ Variable, welche die Anzahl der genutzten Kerne spezifiziert. Im Folgenden ist näher ausgeführt, wo diese eine Rolle spielt.

1. MLMC Simulation

- main.py
 - Das ist das main File, um die Simulation zu starten.
 - Einstellbar sind die Parameter „error_bound“ und „jump_rates“. Diese geben vor, für welche Parameterkombinationen die Simulationen gerechnet werden.
 - Das File speichert results .feather Files für jede Kombination aus error, jump rate, Options Typ und Diskretisierungsverfahren (fixed grid, jump adapted, cutoff).
 - Ebenso werden die Kosten und Preise in .feather files für alle error und jump rate Kombinationen gespeichert. Dabei wird jeweils für die Kosten und die Preise pro Optionstyp und Diskretisierungsverfahren ein File erstellt.
 - Ferner sind zu Beginn der main() Funktion die Parameter der Optionen und des Sprungmodells einstellbar.
- mlmc.py
 - Hier liegen alle Funktionen, um den MLMC Algorithmus auszuführen.
 - Die Hauptfunktion heißt mlmc_main() und wird von main.py aufgerufen.
 - Da die einzelnen Samples parallelisiert berechnet werden, gibt es eine „workers“ Variable. Diese gibt an, wie viele (logische) Kerne genutzt werden. Diese ist je nach PC anzupassen.
- discretization.py
 - Hier liegen die Funktionen, um Diskretisierungen des Preisprozesses zu erzeugen.
 - Dabei gibt es jeweils für die asiatische und Barrier Option eine Diskretisierungsfunktion für das Festgitter- und für das sprungadaptierte Verfahren. Für das Cutoffverfahren wird die Funktion für das sprungadaptierte Verfahren wieder verwendet.
- options.py
 - Hier handelt es sich um ein class-file, da jede Option als Klasse verstanden wird. Die Klassenstruktur dient insbesondere als Parameter und Funktionen Speicher.

- Barrier und asiatische Optionen sind jeweils Unterklassen der Klasse „Option“.
- Beide Unterklassen haben ihre eigenen Auszahlungsmethoden für den groben und feinen Pfad.
- merton_jump_model.py
 - Hier handelt es sich auch um ein class-file.
 - Die Jump Model Klasse dient zum einen als Parameter Speicher und hat Methoden zum Erzeugen von Sprungzeiten und -höhen.

2. MC Simulation

- monte_carlo.py
 - Dieses File kann genutzt werden, um die Preise für die Optionen und Diskretisierungsverfahren mittels einer Monte Carlo Simulation zu schätzen (dient vor allem zur Überprüfung der Implementierung des MLMC Algorithmus).
 - Einstellbar hier sind neben den Modell- und Optionsparameter vor allem die Sprungraten, für welche Preise bestimmt werden.
 - Die Preise werden am Ende für jede Option, Diskretisierungsverfahren und Sprungrate in einem einzelnen .feather File gespeichert.
 - Da die Berechnung der Samples hier auch Parallelisierung nutzt, gibt es wieder eine „workers“ Variable, die je nach Anzahl der Kerne einzustellen ist.
 - Das File nutzt die Skripte option.py, merton_jump_model.py und discretization.py.

3. Schätzung der Konvergenzraten für den erwarteten Samplefehler und die Samplevarianz

- mc_rates_estimates.py
 - Dieses File ist dazu da, um mittels einer Monte Carlo Simulation den erwarteten Samplefehler und die Samplevarianz zu schätzen.
 - Dazu ist im Vorfeld das maximale Level („max_level“) zu setzen (Standard ist 10).
 - Ferner ist festzulegen, für welche Sprungraten die Simulationen ausgeführt werden.
 - Die einzelnen Werte pro Level werden pro Option, Verfahren und Sprungrate jeweils in einem .feather File gespeichert.
 - Am Ende wird pro Konvergenzrate ein .feather File gespeichert, welches die geschätzten Raten für jede Option mit jedem Verfahren pro Sprungrate speichert.

- Da die Berechnung der Samples hier auch Parallelisierung nutzt, gibt es wieder eine „workers“ Variable, die je nach Anzahl der Kerne einzustellen ist.
- Das File nutzt die Skripte option.py, merton_jump_model.py und discretization.py.

4. Plots

Zur Erstellung der Plots gibt es 4 Skripte, welche unabhängig von einander ausgeführt werden können. Allerdings ist erforderlich, dass die Ergebnisse der vorherigen 3 Teile vorliegen und gespeichert wurden. Ebenso enthält jedes Plot Skript entweder ein array mit den gerechneten Fehlerniveaus oder Sprungraten (oder beides). Diese Arrays müssen mit denen aus den vorherigen Programme übereinstimmen.

- plots_price.py
 - Erstellt Plots, welche die MLMC Preise mit den MC Benchmark Preise vergleicht.
- prices_costs.py
 - Erstellt Plots, um die benötigten Kosten der MLMC Simulationen zu vergleichen.
- plots_samples.py
 - Erstellt Plots, um die benötigte Anzahl Samples pro Level zu vergleichen
- plots_estimates.py
 - Erstellt Plots, um die Ergebnisse aus mc_rates_estimates.py zu vergleichen.

Dependencies

- Pandas
- Numpy
- Scipy
- Matplotlib
- ggf. Pyarrow